



CompSci 130

Introduction to Software Fundamentals

Algorithm Analysis/Complexity



Agenda & Reading

▶ Agenda and Learning Outcome:

- ▶ Comparing the efficiency of algorithms
- ▶ Explain running time through counting statements/operations
- ▶ Big-O notation to describe algorithm efficiency
- ▶ Calculating Big-O
- ▶ Properties of Big-O
- ▶ Growth Rate Examples

▶ Reading:

- ▶ Online Textbook: <https://onlinetextbook.auckland.ac.nz/3-efficiency-complexity/>



1 Introduction

Algorithm Analysis/Complexity

- ▶ When we analyze the **performance** (or efficiency) of an algorithm, we are interested in how **much** of a given **resource** the algorithm uses to solve a problem.
- ▶ The most common performance related resources are **time** (how many steps/instructions it takes to solve a problem) and **space** (how much memory it takes).
- ▶ In practice we are going to be mainly interested in **how long** our programs take to **run**, as time is generally a more precious resource than space.



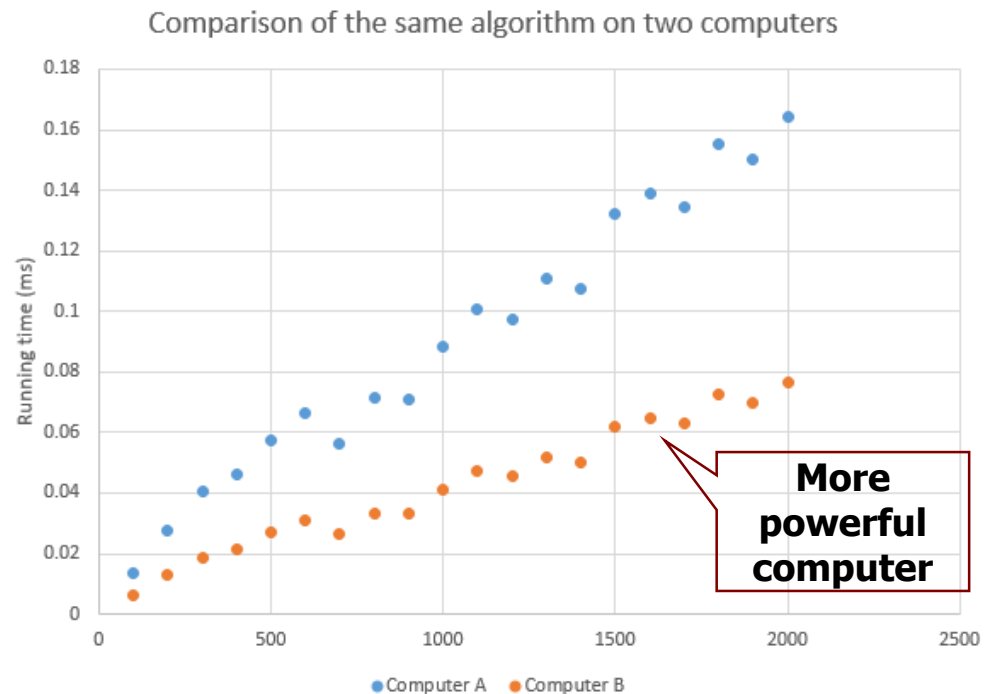
1 Introduction

Efficiency of Algorithms

- ▶ The following graphs show the execution time, in milliseconds, of a given problem executed on **different computers** and plotted against size **n** of the problem

- ▶ We note: the actual running time of a program depends not only on the efficiency of the algorithm, but on many other factors:

- ▶ Processor speed & type
- ▶ Operating system
- ▶ Hard disk vs SSD access
- ▶ Network bandwidth, etc...





1 Introduction

Running-time of Algorithms

- ▶ In order to **compare algorithm runtime** experimentally
 - ▶ All other factors must be kept constant, i.e.
 - ▶ independent of **specific implementations**,
 - ▶ independent of **computers** used, and,
 - ▶ independent of the **data** on which the program runs
 - ▶ This experimentation would involve a lot of work taking different combinations into account to figure out which algorithm is faster
 - ▶ It is better to have theoretical means of arguing about algorithm runtime, independent of other factors
 - ▶ Theoretically, we are often interested in algorithm behaviour **in the worst case**, i.e. given the data of size n that leads to the longest runtime
 - ▶ E.g. searching for a specific item in a shopping cart may require you to pick up all items in the cart...





1 Introduction

Example 1

► Task:

- Complete the `sum_of_n()` function which calculates the sum of the first n natural numbers.
 - **Argument:** an integer
 - **Returns:** the sum of the first n natural numbers as an integer

► Cases:

`sum_of_n(5)`

15

`sum_of_n(100000)`

5000050000



1 Introduction

Algorithm 1

► `sum_of_n_slow`

set `the_sum = 0`

add each value to `the_sum` using
a for loop

return `the_sum`

```
def sum_of_n_slow(n):  
    the_sum = 0  
    for number in range(1, n + 1):  
        the_sum = the_sum + number  
    return the_sum
```

```
time_start = time.time()  
the_sum = sum_of_n_slow(5)  
time_end = time.time()  
print(time_end - time_start, the_sum)
```

The timing calls
embedded before and
after the function call
are used to calculate
the time required for
the calculation (we
must “**import time**”
to use this)



1 Introduction

Algorithm 2

► sum_of_n_fast

set the_sum = 0

use the equation $(n(n + 1))/2$, to calculate the total

return the_sum

```
def sum_of_n_fast(n):  
    the_sum = (n * (n + 1)) // 2  
    return the_sum  
  
time_start = time.time()  
the_sum = sum_of_n_fast(5)  
time_end = time.time()  
print(time_end - time_start, the_sum)
```



1 Introduction

Experimental Result

- ▶ Using 4 different values for n: [10000, 100000, 1000000, 10000000]

n	sum_of_n_slow (for loop)	sum_of_n_fast (equation)
10000	0.0033 s	0.00000181 s
100000	0.0291 s	0.00000131 s
1000000	0.3045 s	0.00000107 s
10000000	2.7145 s	0.00000123 s

Time
Consuming
Process!

Time increases as we
increase the value of n.

NOT impacted by
the number of
integers being added.

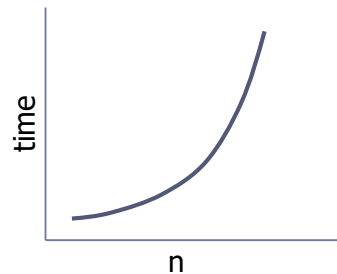
- ▶ We have to **count** the number of basic operations of an algorithm, and then **generalise** this count.



1 Introduction

Experimental Result

- ▶ In general, we could plot the running times when comparing two algorithms and we may see different shapes:

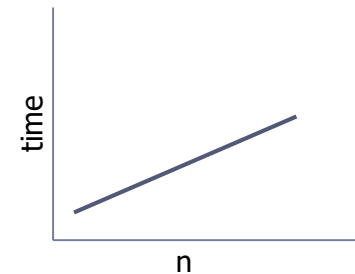


$$\text{time} = c \cdot n^2$$

Plot is a parabola

quadratic

$$O(n^2)$$



$$\text{time} = c \cdot n$$

Plot is a straight line

linear

$$O(n)$$

We will learn
about this
terminology
today

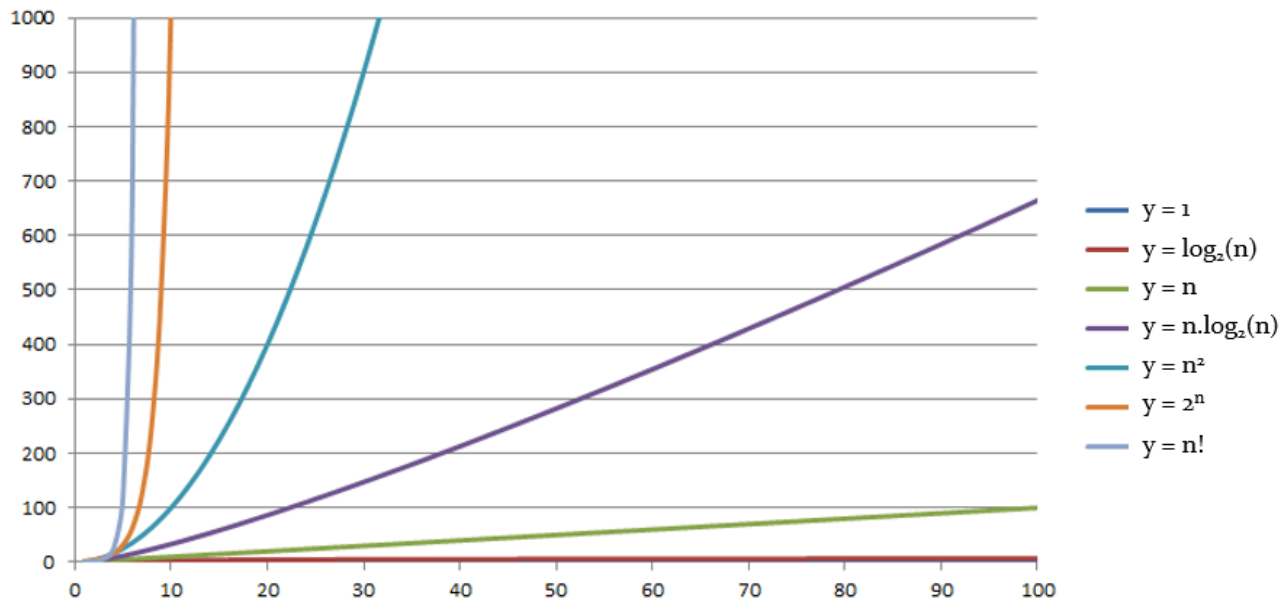




1 Introduction

Experimental Result

- ▶ In general, we could plot the running times when comparing two algorithms and we may see different shapes:



We will see there are some typical “shapes” that we talk about often!



2 Counting Operations

Basic Operations

- ▶ We need to **estimate** the running time as a function of the problem size n .
- ▶ A **primitive operation** takes **a unit of time**. The actual length of time will depend on external factors such as the hardware and software environment.
 - ▶ Each of these kinds of operation would take approximately the same amount of time on a given hardware and software environment.
 - ▶ Assigning a value to a variable.
 - ▶ Calling a method.
 - ▶ Performing an arithmetic operation.
 - ▶ Comparing two numbers.
 - ▶ Indexing a list element.
 - ▶ Returning from a function.



2 Counting Operations

Example 2A

- ▶ Example: Calculating the sum of the first 10 non-negative numbers, i.e. $0 + 1 + 2 + 3 + \dots + 9 \Rightarrow 45$

```
def sum_first_10_non_negative_numbers():  
    the_sum = 0  
    index = 0  
    while index < 10:  
        the_sum += index  
        index += 1  
    return the_sum
```

1 assignment ->
1 assignment ->
11 comparisons ->
10 plus/assignments ->
10 plus/assignments ->
1 return ->

- ▶ Total = 34 operations
- ▶ Counting the number of primitive operations:
 - ▶ For each line of pseudocode, count the number of primitive operations in it.
 - ▶ Multiply this count by the number of times this line is executed.
 - ▶ Sum up over all lines.



2 Counting Operations

Example 2B

- ▶ Example: Calculating the sum of n non-negative numbers
 - ▶ i.e. $0 + 1 + 2 + \dots + n$

1 assignment ->
1 assignment ->
 $n + 1$ comparisons ->
 n plus/assignments ->
 n plus/assignments ->
1 return

```
def sum_non_negative_numbers_up_to(n):  
    the_sum = 0  
    index = 0  
    while index < n:  
        the_sum += index  
        index += 1  
    return the_sum
```

- ▶ Total = $3n + 4$ operations

Exercise 1 (Basic Skill)

- ▶ Select the most appropriate run-time formulas (in terms of primitive operations needed for input of size n):

num = 0, 2, 4, 6, 8,
10, 12, 14, 16, 18
=> number of iterations = 10

Note that a loop condition is checked 1 time more than the loop body is executed.

num = n , $n-1$, $n-2$, ... 2, 1
=> number of iterations = n

```
def rate():  
    num = 0  
    the_sum = 0  
    while num < 20:  
        the_sum += num  
        num += 2
```

- A) $f(n) = 22$
- B) $f(n) = 33$
- C) $f(n) = 30$
- D) $f(n) = 21$

```
def rate(n):  
    num = n  
    the_sum = 0  
    while num > 0:  
        the_sum += num  
        num -= 1
```

- A) $f(n) = 3n + 3$
- B) $f(n) = 3n$
- C) $f(n) = 33$
- D) $f(n) = 2n + 2$



2 Counting Operations

Problem size

- ▶ Performance is usually measured by the **rate** at which the running time increases as the problem size gets bigger,
 - ▶ i.e. we are interested in the relationship between the **running time** and the **problem size**.
 - ▶ It is very important that we clearly **identify what the problem size is**.
 - ▶ For example, if we are analyzing an algorithm that processes a list, the problem size is probably the **number of elements** in the list.
 - ▶ Sometimes, the problem size will simply be the **value** of a variable, where the running time of the program depends on how big that value is. Think about summing the first n integers in our **sum_of_n** example
- ▶ In other words, we want to know how **scalable** our solution is, i.e. how well it performs under growing problem size n



2 Counting Operations

Example 3

► Consider the following two algorithms:

► Algorithm A:

- Outer Loop: n iterations
- Inner Loop: $\frac{n}{5}$ iterations
- **Total** = $(n * \frac{n}{5}) = (\frac{n^2}{5})$ iterations

```
for x in range(0, n):  
    for y in range(0, n, 5):  
        ...
```

► Algorithm B:

- Outer Loop: n iterations
- Inner Loop: 5 iterations
- **Total** = $n * 5 = 5*n$ iterations

```
for x in range(0, n):  
    for y in range(0, 5):  
        ...
```

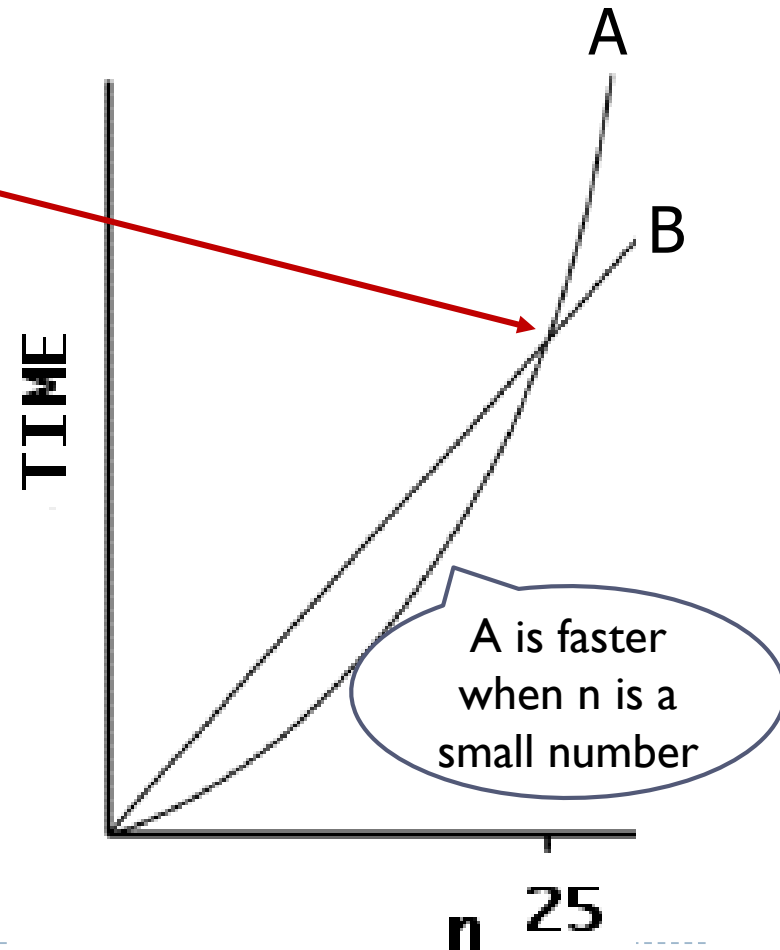


2 Counting Operations

Growth Rate Function – A or B?

n	5	10	15	20	24	25	26	30
A	5	20	45	80	115	125	135	180
B	25	50	75	100	120	125	130	150

- ▶ If n is 10^6 ,
 - ▶ Algorithm A's time requirement is
 - ▶ $(\frac{n^2}{5}) = (\frac{10^{12}}{5}) = 2 * 10^{11}$
 - ▶ Algorithm B's time requirement is
 - ▶ $5 * n = 5 * 10^6$
- ▶ What does the growth rate tell us about the running time of the program?





2 Counting Operations

Growth Rate Function – A or B?

- ▶ For smaller values of n , the differences between algorithm A ($n^2/5$) and algorithm B ($5n$) are not very big
- ▶ But the differences are very significant for larger problem sizes such as for $n > 1,000,000$
 - ▶ $2 * 10^{11}$ vs $5 * 10^6$
 - ▶ At the same time additive and multiplicative factors become less relevant for large n
- ▶ Our main interest in algorithm analysis is what happens with our runtime efficiency when **n becomes very large**
 - ▶ This is where **efficient** and non-efficient (**intractable**) algorithms are distinguished



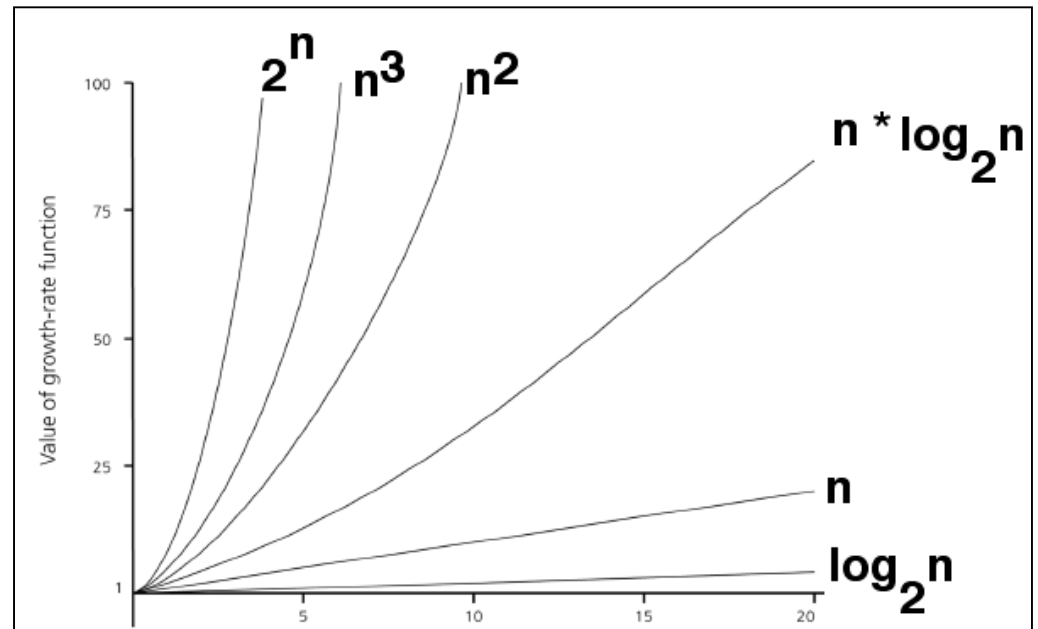
2 Counting Operations

Comparison of Growth Rate

- ▶ We use 'Big-O' notation (capital letter O) to specify the order of complexity of an algorithm (e.g. runtime complexity)
 - ▶ e.g. $O(n)$, $O(n^2)$

If a problem of size n requires time that is directly proportional to n , the algorithm is $O(n)$

If a problem of size n requires time that is directly proportional to n^2 , the algorithm is $O(n^2)$



A comparison of growth-rate functions in graphical form

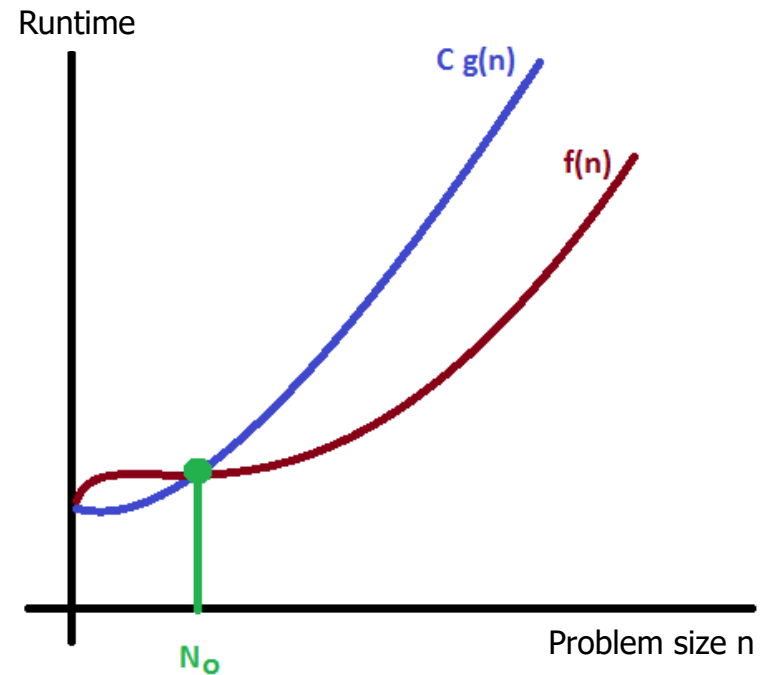


2 Counting Operations

Comparison of Growth Rate

► 'Big-O' notation indicates worst case behaviour

- This means that if we have an algorithm with actual runtime $f(n)$ for varying problem sizes n , we say it has $O(g(n))$ complexity if there exist constants C and N_0 such that $f(n) < C * g(n)$ for all $n > N_0$
- The example indicates that even in the worst case, our algorithm runtime will never be worse than quadratic, with n growing larger!



Example where $g(n)$ is a quadratic function, so $f(n) = O(n^2)$



5 Calculating Big-O

Calculating Big-O

- ▶ Rules for finding out the time complexity of a piece of code
 - ▶ Straight-line code
 - ▶ Loops
 - ▶ Nested loops
 - ▶ Consecutive statements
 - ▶ If-then-else statements
 - ▶ Logarithmic complexity



5 Calculating Big-O Rules

▶ Rule 1: Straight-line code

- ▶ Big-O = Constant time $O(1)$
- ▶ Does not vary with the size of the input
- ▶ Example:
 - ▶ Assigning a value to a variable
 - ▶ Performing an arithmetic operation.
 - ▶ Indexing a list element.

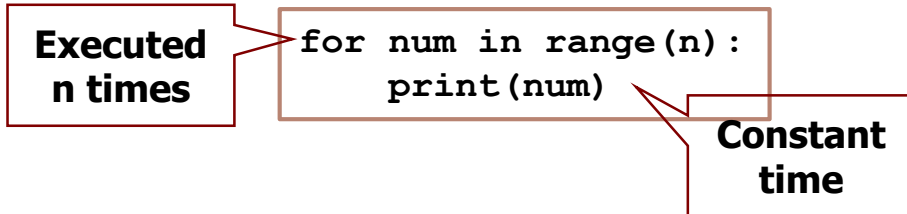
```
value = number1 + number2  
value = 10
```



5 Calculating Big-O Rules

► Rule 2: Loops

- The running time of the statements inside the loop (including tests) times the number of iterations
- Example:
 - Constant time * n
 - $= c * n = O(n)$





5 Calculating Big-O Rules

► Rule 3: Nested Loop

- Analyze inside out. Total running time is the product of the number of iterations performed in all the individual loops.
- Example:
 - $\text{constant} * (\text{inner loop: } n) * (\text{outer loop: } n)$
 - $\text{Total time} = c * n * n = c * n^2 = O(n^2)$

**Outer loop:
Executed n times**

```
for x in range(n):  
    for y in range(n):  
        k = x + y
```

**Inner loop:
Executed n times**



5 Calculating Big-O Rules

► Rule 4: Consecutive statements

- Add the time complexities of each statement
- Ignore low order times by looking for maximum

► Example:

- Constant time + n times * constant time
- $c_0 + c_1 n$
- Big-O = $O(f(n) + g(n))$
- $= O(\max \{ f(n), g(n) \})$
- $= O(n)$

**Executed
n times**

```
x = x + 1  
for i in range(n):  
    m = m + 2;
```

**Constant
time**



5 Calculating Big-O Rules

► Rule 5: if-else statement

- Worst-case running time: the test, plus either the `if` part or the `else` part (whichever is the larger).
- Example:
 - $c_0 + \max\{c_1, (n * (c_2 + c_3))\}$
 - Total time = $c_0 + n(c_2 + c_3) = O(n)$
- Assumption:
 - The condition can be evaluated in constant time. If this is not possible, we need to add the time to evaluate the expression.

Test:
Constant time c_0

```
if len(a) != len(b):  
    return False  
else:  
    for index in range(len(a)):  
        if a[index] != b[index]:  
            return False
```

True case:
Constant c_1

False case:
Executed n times

Another if:
constant c_2 + constant c_3



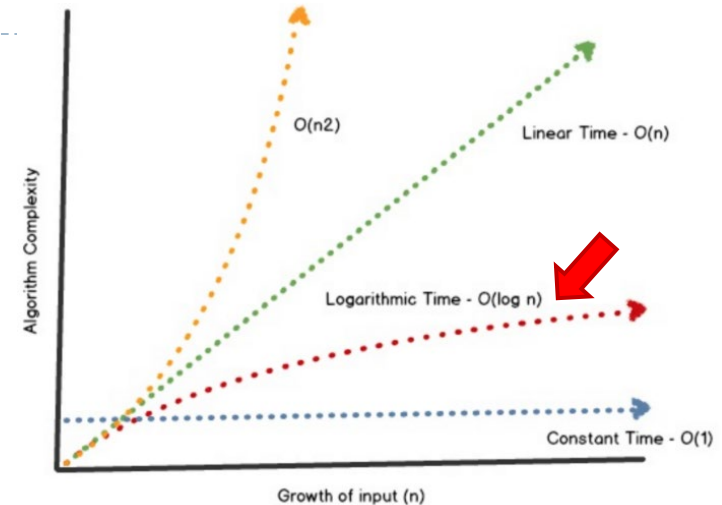
5 Calculating Big-O Rules

► Rule 6: Logarithmic

- An algorithm is $O(\log n)$ if it takes a constant time to cut the problem size by a fraction (usually by $\frac{1}{2}$)

► Example:

- Finding a phone number in an alphabetically sorted phone book of n pages
 - Look at the centre page in the phone book
 - Is the person's name before or after the top name of this page?
 - Repeat process with left or right part of phone book until the person's name is found. Each time we check for a name, we **halve** the number of pages that we have to look at!





5 Calculating Big-O Rules

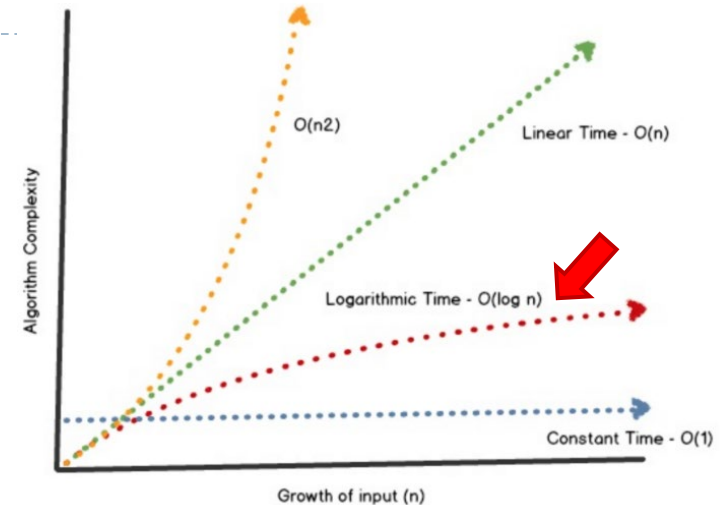
► Rule 6: Logarithmic

- An algorithm is $O(\log n)$ if it takes a constant time to cut the problem size by a fraction (usually by $1/2$)

- Example:

```
size = n
while size > 1:
    // O(1) stuff
    size //= 2
```

- Size: $n, n/2, n/4, n/8, n/16, \dots, 2, 1$
- If $n = 2^k$, it would be approximately k steps. The loop will execute $\log k$ times in the worst case ($\log_2 n = k$). Big-O = $O(\log n)$
- Note: For Big-O notation we don't need to indicate the base. The logarithms to different bases differ only by a constant multiplicative factor, e.g. $\log_2 n = \log_{10} n / \log_{10} 2$





Exercise 2 (Basic Skill)

- For the following blocks of code, select the most appropriate run-time formulas:

number = 1, 2, 4
=> number of iterations = 3

```
def rate():  
    number = 1  
    total = 0  
    while number < 8:  
        total += number  
        number *= 2  
    return total
```

- A) $f(n) = 16$
- B) $f(n) = 13$
- C) $f(n) = 10$
- D) $f(n) = 8$

number = 8, 4, 2
=> number of iterations = 3

```
def rate():  
    number = 8  
    total = 0  
    while number > 1:  
        total += number  
        number //= 2  
    return total
```

- A) $f(n) = 16$
- B) $f(n) = 13$
- C) $f(n) = 10$
- D) $f(n) = 8$



6 Growth Rate Examples

Hypothetical Running Time

- ▶ The running time on a hypothetical computer that computes 10^6 operations per second for various problem sizes

Notation		n 10	10^2	10^3	10^4	10^5	10^6
$O(1)$	Constant	1 μ sec	1 μ sec	1 μ sec	1 μ sec	1 μ sec	1 μ sec
$O(\log(n))$	Logarithmic	3 μ sec	7 μ sec	10 μ sec	13 μ sec	17 μ sec	20 μ sec
$O(n)$	Linear	10 μ sec	100 μ sec	1 msec	10 msec	100 msec	1 sec
$O(n\log(n))$	N log N	33 μ sec	664 μ sec	10 msec	13.3 msec	1.6 sec	20 sec
$O(n^2)$	Quadratic	100 μ sec	10 msec	1 sec	1.7 min	16.7 min	11.6 days
$O(n^3)$	Cubic	1 msec	1 sec	16.7 min	11.6 days	31.7 years	31709 years
$O(2^n)$	Exponential	10 msec	3e17 years				

Intractable problems



Properties of Big-O

- ▶ There are three properties of Big-O when combining terms
 - ▶ Ignore **low-order terms** in the function (smaller terms)
 - ▶ $f(n) + g(n) = O(\max\{f(n), g(n)\})$
 - ▶ e.g. $n^3 + n^2 + \log(n)$ is $O(n^3)$
 - ▶ Ignore any **constants** in the high-order term of the function
 - ▶ $C \times f(n) = O(f(n))$
 - ▶ e.g. $100n$ is $O(n)$
 - ▶ **Combine** growth-rate functions
 - ▶ If $f(n)$ is $O(p)$ and $g(n)$ is $O(q)$ then $f(n) \times g(n)$ is $O(p \times q)$
 - ▶ e.g. two nested loops: $n \times n = O(n^2)$
 - ▶ e.g. $n^2 \times \log(n) \times n$ is $O(n^3 \log(n))$



Ignore low order terms

- ▶ Consider the function: $f(n) = n^2 + 100n + \log_{10} n + 1000$
 - ▶ For small values of n the last term, 1000, dominates.
 - ▶ When n is around 10, the terms $100n + 1000$ dominate.
 - ▶ When n is around 100, the terms n^2 and $100n$ dominate.
 - ▶ When n gets much larger than 100, the n^2 dominates all others.
- ▶ So it would be safe to say that this function is $O(n^2)$ for values of $n > 100$

- ▶ More examples:

- ▶ $f(n) = n^3 + n^2 + n + 5000$ Big-O is $O(n^3)$

- ▶ $f(n) = n + n^2 + 5000$ Big-O is $O(n^2)$



Ignore any constants

- ▶ Consider the function:

$$f(n) = 254 * n^2 + n$$

- ▶ Big-O is $O(n^2)$
- ▶ Consider another function:

$$f(n) = n / 30$$

- ▶ Big-O is $O(n)$
- ▶ And consider another function:

$$f(n) = 3n + 1000$$

- ▶ Big-O is $O(n)$



Combine functions

- ▶ Consider the function:

$$f(n) = n * \log n$$

- ▶ Big-O is $O(n \log n)$

- ▶ Consider another function:

$$f(n) = n^2 * n$$

- ▶ Big-O is $O(n^3)$

Exercise 3

► Programming Question:

► Modify the function:

► Calculate the **total number of operations**

n=4, number of operations = 16
n=7, number of operations = 25

```
def sum_numbers(n):  
    total = 0  
    number = 0  
    while number < n:  
        total += number  
        number += 1  
    return total
```

when n = 4
2 steps
Loop = 3 steps * 4 times + 1 comparison
1 return
= 16 steps

```
def sum_numbers(n):  
    total = 0  
    number = 0  
    count = 2 + 1  
    while number < n:  
        count += 3  
        total += number  
        number += 1  
    count += 1  
    print(...format(count))  
    return total
```

count = 2 + 1
i.e. two steps at the beginning
one step for extra comparison

count += 3
two steps inside the loop
one for comparison

count += 1
the return statement

Exercise 4

► Theory Question:

2 steps

Loop = 3 steps * n iterations + 1 comparison

1 return

= $3n + 4$

Formula is $3n + 4$

when n is 7, the number of operations is 25

when n is 10, the number ... is 34

when n is 100, the number ... is 304

Fill in the blanks

```
def sum_numbers(n):  
    total = 0  
    number = 0  
    while number < n:  
        total += number  
        number += 1  
    return total
```

Test	Expected Result
rate(7)	Number of operations:
rate(10)	
rate(100)	



Exercise 5

- ▶ What is the Big-O of the following statements?

outer loop:
Execute n
times

```
number = 0
for i in range(n):
    for j in range(10):
        number += i
```

inner loop:
Execute 10 times

Constant time

- ▶ 1 step
- ▶ Outer Loop: n iterations
- ▶ Inner Loop: 10 iterations
- ▶ statement inside the inner loop = constant
- ▶ Total = $1 + n * 10 * 1$
- ▶ Big-O =

?



Exercise 6

- ▶ What is the Big-O of the following consecutive statements?

**n
iterations**

```
number = 0
for i in range(n):
    for j in range(n):
        number += i
    for k in range(n):
        number += k
```

**n
iterations**

- ▶ 1 step
- ▶ $n * n * 1$
- ▶ $n * 1$
- ▶ Total = $1 + n^2 + n$
- ▶ Big-O =

?



Exercise 7

- ▶ What is the Big-O of the following function?

```
def rate(n):  
    i = 0  
    total = 0  
    while i < 5:  
        j = 1  
        while j < n:  
            total += j  
            j *= 2  
        i += 1  
    return total
```

5
iterations

log(n)
iterations

Big-O = ?



Exercise 8

- ▶ What is the Big-O of the following statements?

```
...  
for i in range(n):  
    for j in range(i+1, n):  
        number += i
```

- ▶ Calculate the number of iterations for the nested loop:
 - ▶ When i is 0, the inner loop executes $(n-1)$ times. (i.e. j in $\text{range}(1, n)$)
 - ▶ When i is 1, the inner loop executes $n-2$ times. (i.e. j in $\text{range}(2, n)$)
 - ▶ ...
 - ▶ When i is $n-2$, the inner loop executes once.
- ▶ The number of times the inner loop statements execute:
 - ▶ $(n-1) + (n-2) \dots + 2 + 1$
 - ▶ $= n * (n-1) / 2$
- ▶ Big-O =

Math formula



Caution!

- ▶ Big-O notation gives us a convenient way to “talk about” the efficiency of algorithms. It is a common notation, and widely understood.
- ▶ However it is not perfect – for example, the constant factors sometimes can be significant. Consider this example:

Running time of Algorithm A, $T_A = 20n$

Running time of Algorithm B, $T_B = 0.1 * n * \log(n)$

Which algorithm is more efficient?



Caution!

- ▶ Big-O notation gives us a convenient way to “talk about” the efficiency of algorithms. It is a common notation, and widely understood.
- ▶ However it is not perfect – for example, the constant factors sometimes can be significant. Consider this example:

Algorithm A

$$T_A = 20n$$

This is $O(n)$

Algorithm B

$$T_B = 0.1n \log(n)$$

This is $O(n \log n)$

So according to Big-O, Algorithm A is more efficient!



Caution!

- ▶ Big-O notation gives us a convenient way to “talk about” the efficiency of algorithms. It is a common notation, and widely understood.
- ▶ However it is not perfect – for example, the constant factors sometimes can be significant. Consider this example:

Algorithm A

$$T_A = 20n$$

This is $O(n)$

Algorithm B

$$T_B = 0.1n \log(n)$$

This is $O(n \log n)$

But in practice, $20n > 0.1n \log(n)$ until $n \approx 2^{200} = 10^{60}$!

So in all practical cases, Algorithm B is actually better...



Summary

- ▶ Complexity analysis measures an algorithm's time requirement as a function of the problem size by using a growth-rate function.
 - ▶ It is an **implementation-independent** way of measuring an algorithm
- ▶ Complexity analysis focuses on **large** problems
- ▶ Worst-case analysis considers the **maximum** amount of work an algorithm will require on a problem of a given size
 - ▶ Generally we want to know the worst-case running time.
 - ▶ It provides the upper bound on time requirements
 - ▶ Often average and sometimes best case complexity is also of interest



Additional Material

Examples

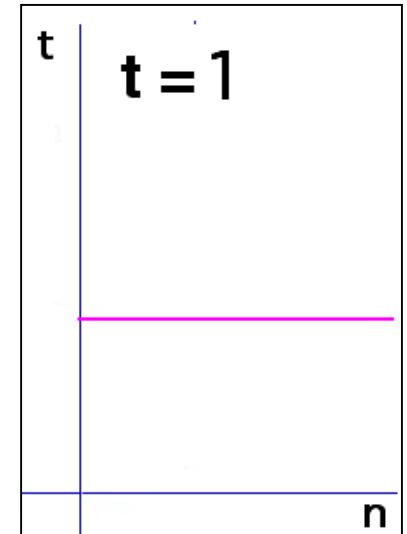


6 Growth Rate Examples

Constant Growth Rate - $O(1)$

- ▶ Time requirement is constant and, therefore, independent of the problem's size n .

```
def constant_time(n):  
    s = "start assignments the day before they are due"  
    for num in range(25):  
        print("I must not ", s)
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(1)$	1	1	1	1	1	1

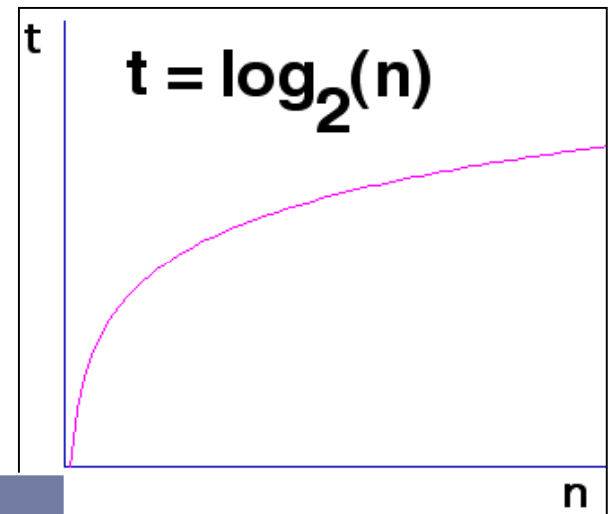


6 Growth Rate Examples

Logarithmic Growth Rate - $O(\log n)$

- ▶ Increase slowly as the problem size increases
- ▶ If you square the problem size, you only double its time requirement
- ▶ The base of the log does not affect a log growth rate, so you can omit it.

```
def logarithmic_time(n):  
    s = "YELL"  
    num = 1  
    while num < n:  
        print("I must not ", s)  
        num = num * 2
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(\log_2 n)$	3	6	9	13	16	19

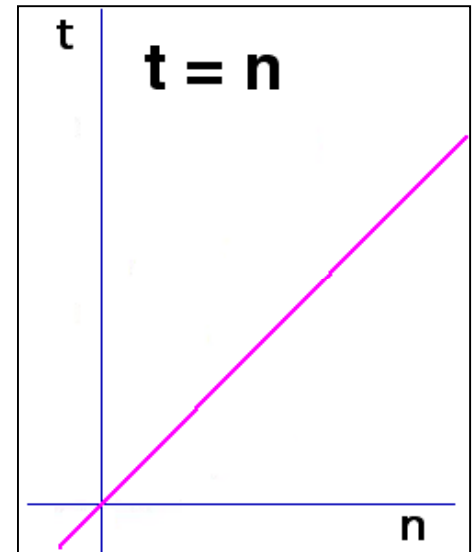


6 Growth Rate Examples

Linear Growth Rate - $O(n)$

- ▶ The time increases directly with the size of the problem.
- ▶ If you square the problem size, you also square its time requirement

```
def linear_time(n):  
    s = "FIGHT"  
    for num in range(n):  
        print("I must not ", s)
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(n)$	10	10^2	10^3	10^4	10^5	10^6

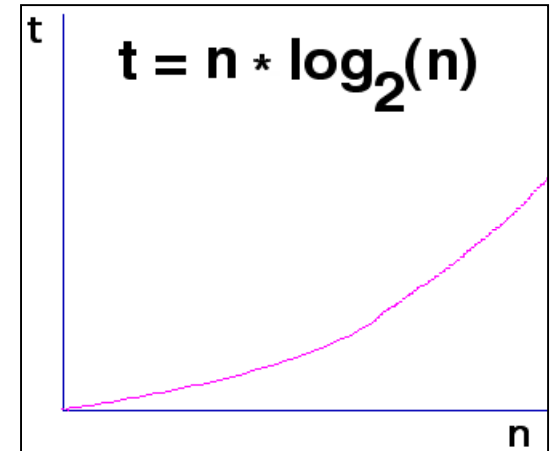


6 Growth Rate Examples

$n \cdot \log n$ Growth Rate - $O(n \log(n))$

- ▶ The time requirement increases more rapidly than a linear algorithm.
- ▶ Such algorithms usually divide a problem into smaller sub-problems that are each solved separately.

```
def quasi_linear_time(n):  
    s = "HIT"  
    for x in range(n):  
        y = n  
        while y > 1:  
            print("I must not ", s)  
            y = y // 2
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(n \log(n))$	30	664	9965	10^5	10^6	10^7

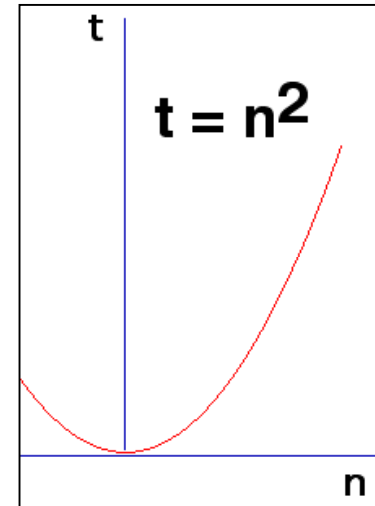


6 Growth Rate Examples

Quadratic Growth Rate - $O(n^2)$

- ▶ The time requirement increases rapidly with the size of the problem.
- ▶ Algorithms that use two nested loops are often quadratic.

```
def quadratic_time(n):  
    s = "LIE"  
    for x in range(n):  
        for y in range(n):  
            print("I must not ", s)
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(n^2)$	10^2	10^4	10^6	10^8	10^{10}	10^{12}

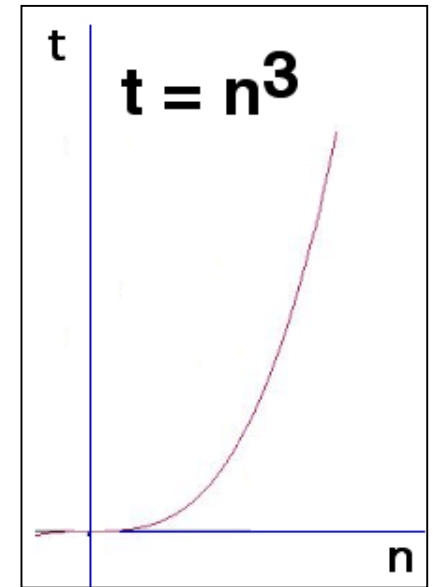


6 Growth Rate Examples

Cubic Growth Rate - $O(n^3)$

- ▶ The time requirement increases more rapidly with the size of the problem than the time requirement for a quadratic algorithm
- ▶ Algorithms that use three nested loops are often cubic and are practical only for small problems.

```
def cubic_time(n):  
    s = "SULK"  
    for x in range(n):  
        for y in range(n):  
            for z in range(n):  
                print("I must not ", s)
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(n^3)$	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}

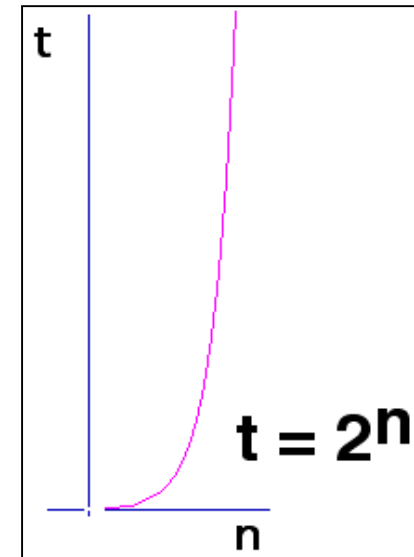


6 Growth Rate Examples

Exponential Growth Rate - $O(2^n)$

- ▶ As the size of a problem increases, the time requirement usually increases too rapidly to be practical. Such algorithms are called intractable.

```
def exponential_time(n):  
    s = "POKE OUT MY TONGUE"  
    for x in range(2 ** n):  
        print("I must not ", s)
```



n	10^1	10^2	10^3	10^4	10^5	10^6
$O(2^n)$	10^3	10^{30}	10^{301}	10^{3010}	10^{30103}	10^{301030}